# 仿真可视化工具（tools/）

`plot_sim.py` 读取 `ar_sim` 导出的 CSV，生成多子图分析图与机动极坐标轨迹图。 `animate_sim.py` 读取 CSV，生成时间演化的动画视频（MP4/GIF）。 `plot_convergence.py` 读取 `ar_sim --tune` 导出的收敛轨迹 CSV，生成 RLS 辨识收敛、 ESC 增益/J 收敛与全链路概览三张效果图（见 `docs/auto_tuning/自动调参测试用例.md` §6.4）。

## 依赖

系统已安装： `python3-matplotlib` 、 `python3-numpy` 、 `ffmpeg` 。

```
# 若缺失，可一键安装
sudo apt-get install -y python3-matplotlib python3-numpy ffmpeg
```

## 一键流程：跑场景 → 导出 CSV → 生成图

```
cd control
mkdir -p out

# 1) 30° 阶跃（典型船参）
./build/sim/ar_sim --scenario step_typical --out out/step.csv
python3 tools/plot_sim.py out/step.csv -o out/step.png --metrics

# 2) RLS 闭环辨识
./build/sim/ar_sim --ident --out out/ident.csv
python3 tools/plot_sim.py out/ident.csv -o out/ident.png

# 3) ESC 长期
./build/sim/ar_sim --esc --out out/esc.csv
python3 tools/plot_sim.py out/esc.csv -o out/esc.png

# 4) 六机动形态（带极坐标轨迹）
./build/sim/ar_sim --maneuver williamson --out out/will.csv
python3 tools/plot_sim.py out/will.csv -o out/will.png --polar

# 5) 全验证矩阵（自动按 case 分文件导出）
./build/sim/ar_sim --validate --out out/validate.csv
python3 tools/plot_sim.py out/validate_*.csv --grid -o out/validate.png --metrics

# 6) 三船参阶跃叠加对比
./build/sim/ar_sim --all --out out/all.csv
python3 tools/plot_sim.py out/all_step_typical.csv out/all_step_fast.csv out/all_step_slow.csv
```

```
\
    -o out/compare.png --metrics

# 7) 自动调参收敛试验 + 收敛效果图（U24：典型船全链路收敛）
./build/sim/ar_sim --tune --K 0.2 --T 8 --seconds 1200 --out out/tune.csv
python3 tools/plot_convergence.py out/tune.csv --trueK 0.2 --trueT 8 -o out/figures
```

## 调参收敛试验（ar_sim --tune）

`--tune` 跑自动调参全链路收敛试验：Z 形参考激励 → RLS 辨识收敛 → 极点配置建立基线 → ESC 窗口寻优。CSV 每 1 s 记录 $\hat{K}/\hat{T}$、去载波持久增益（KpCenter/KdCenter）、实际 PD 增益， 窗口结算行（settle=1）携带窗口 ITAE J 与结算前后增益。收敛判据：

- 基线建立且辨识 `valid()`；
- 基线后 ≥ 4 个 ESC 窗口，末 3 窗 J 相对散布 ≤ 10%；
- 末窗 J ≤ 首窗 J × 1.02；
- $\hat{K}/\hat{T}$ 相对真值偏差 ≤ 25%，全程数值有界。

```
./build/sim/ar_sim --tune --K 0.2 --T 8 --seconds 1200 --out out/tune.csv
python3 tools/plot_convergence.py out/tune.csv --trueK 0.2 --trueT 8 -o out/figures
```

慢船激励适配：`--leg 60` 可把 Z 形参考翻转周期从默认 15 s 放大到 60 s（建议 ≥ 2T）。

## 子图说明（多子图模式）

| 子图 | 内容 |
| --- | --- |
| 1 | 航向 ψ 与参考 ψ_ref（°，时间） |
| 2 | 舵指令 δ_cmd 与实际 δ_act（含 ±35° 限幅虚线） |
| 3 | 转首率 r（°/s） |
| 4 | 航向误差 e（°） |
| 5 | 增益 Kp / Kd（双 y 轴） |

模式切换处用绿色点划竖线标注。

### 极坐标模式（`--polar`）

以航向 ψ 为极角、累计转首角为半径，绘制机动轨迹。零度朝北、顺时针，符合航海习惯。 起点（绿）、终点（红）单独标注。适合 williamson / uturn / circles / cloverleaf / search 等机动形态。

### 命令行选项

```
csv                 一个或多个 CSV 路径
-o, --out PATH      输出图片（PNG/PDF/SVG）；不指定则弹窗
--polar             追加机动极坐标图（单文件时输出 <stem>_polar.<ext>）
--no-multipane      不画多子图（仅 --polar 时有用）
--grid              多文件网格平铺
--metrics           打印指标表（超调/调节时间/稳态1σ/操舵频次）
--title TEXT        图总标题
```

### 输入输出环境变量

- `PLOT_SIM_AGG=1`：强制使用 Agg 后端（无 GUI 环境，仅保存图片）。

## 动画视频工具（animate_sim.py）

`animate_sim.py` 读取 CSV，生成时间演化的动画视频。左半部分为多子图（航向/参考、舵角、转首率、误差、增益），右半部分可选机动极坐标轨迹。

### 一键生成所有功能仿真视频

```
bash tools/gen_all_videos.sh
```

该脚本会自动跑所有场景（step/ident/esc/六机动/validate），导出 CSV 并生成对应 MP4 到 `out/`。

### 单独生成

```
# 阶跃
python3 tools/animate_sim.py out/step.csv -o out/step.mp4 --fps 25 --speed 2.0
```

```
# 机动 + 极坐标
python3 tools/animate_sim.py out/will.csv -o out/will.mp4 --polar --fps 25 --speed 2.0

# 无 ffmpeg 时用 GIF
python3 tools/animate_sim.py out/step.csv -o out/step.gif --fps 20
```

## 选项

```
csv                 CSV 路径
-o, --out PATH      输出视频（.mp4 需 ffmpeg，.gif 无需）
--fps N             帧率（默认 30）
--speed X           回放速度倍数（>1 加速，默认 1.0）
--polar             右侧追加机动极坐标轨迹
--title TEXT        视频标题
```

## 输入输出环境变量

- `ANIM_SIM_AGG=1`：强制使用 Agg 后端。

## 输出视频清单

| 视频 | 内容 |
| --- | --- |
| step.mp4 | 30° 阶跃响应（航向/舵角/转首率/误差/增益） |
| ident.mp4 | RLS 闭环辨识（Z 形激励 + 极坐标） |
| esc.mp4 | ESC 长期增益整定 |
| williamson.mp4 | Williamson 机动（极坐标） |
| uturn.mp4 | U 形回转（极坐标） |
| zigzag.mp4 | Z 形机动（极坐标） |
| circles.mp4 | 圆形回转（极坐标） |
| cloverleaf.mp4 | 三叶草机动（极坐标） |
| search.mp4 | 搜索机动（极坐标） |
| validate_*.mp4 | 全验证矩阵 4 个 case |

# 全工况机动视频矩阵（gen_maneuver_matrix.sh）

`gen_maneuver_matrix.sh` 生成船舶所有工况的机动效果视频矩阵，输出到 `out/matrix/`：

- **维度1 船参数**：typical (K=0.2/T=8)、fast (K=0.5/T=3)、slow (K=0.1/T=20)
- **维度2 机动形态**：williamson / uturn / zigzag / circles / cloverleaf / search
- **维度3 干扰**：无干扰 / 带风+浪+噪声干扰

共 3 × 6 × 2 = **36 个工况视频**，命名格式：`<船型>_<机动>[_dist].mp4`，例如 `typical_williamson_dist.mp4`。

```
bash tools/gen_maneuver_matrix.sh
```

每个视频含多子图（航向/舵角/转首率/误差/增益）+ 极坐标机动轨迹，标题标注船型与干扰状态。

# 实船调试接口（tuner_debug_api.hpp + ar_tuner_debug）

面向实船调试/标定：`include/tuner_debug_api.hpp` 把 `AutoTuner` 的每个功能摊平为 **可单独调用**的调试方法（经 `friend` 暴露，不改动任何在线行为），`tuner_debug_cli.cpp` 把它们包装为逐行命令工具 `ar_tuner_debug`——每行一条命令，每条命令恰好调用一个功能，输出 `key=value` 便于上位机/脚本解析，可经 ssh/串口终端在船上直接使用。

> **安全约束**：`force*` 系列（forcegains/forcebase/forcephase/forcegrad/forceenabled）旁路稳定门，仅限受控调试场景；正常航行控制回路严禁接入本接口。

## 构建

```
cd control/build && cmake .. && make ar_tuner_debug   # AR_BUILD_TOOLS 默认 ON
```

## 使用示例

```
$ ./build/tools/ar_tuner_debug
> pole 0.2 8                    # 只算极点配置：OK Kp=1.600000 Kd=8.600000
> check 0.2 8 1.6 8.6 0         # 只验稳定门：OK pass=1
> baseline 0.2 8                # 只建基线：OK established=1
> forceenabled 1                # 单独打开 ESC 门控（调试前置）
> dither 1.6 8.6                # 只打一拍载波：OK Kp=1.600000 Kd=9.030000
> accum 1.0 2.0 1               # 只喂一拍样本（成本/振荡/解调）
> step 1.6 8.6                  # 只做一次窗末梯度结算
> status                        # 全量快照（增益/基线/ESC/窗口/振荡）
> tick 10 9 1 12 0.02 1.0 2.0 1 1 0.2 8   # 跑一拍完整在线编排
> quit
```

脚本化：`./ar_tuner_debug < cmds.txt`，成功输出 `OK ...`、失败输出 `ERR <原因>`。

## 命令分组（help 可查全表）

| 组 | 命令 | 对应功能 |
|---|---|---|
| 观测 | `status` / `j` / `osc` / `params` / `setparam` | 快照 / 当前窗口 ITAE / 实测振荡 / 参数表读 / 单参数改写（复位状态） |
| 层2 | `pole` / `check` / `baseline` / `eval` | 极点配置 / 稳定门 / 建基线 / 辨识评估原语 |
| ESC | `dither` / `accum` / `step` / `settle` / `clamp` | 载波注入 / 样本累积+解调 / 窗末梯度结算 / 窗末裁决 / 限幅 |
| 振荡 | `oscfeed` / `oscreset` | 只喂/清零振荡检测器 |
| 生命周期 | `revert` / `rebase` / `suspend` / `reset` | 回退基线 / ESC 重锚 / 挂起 / 全复位 |
| 主流程 | `tick` | 与在线路径同一函数的一拍编排 |
| 强制注入 | `forcegains` / `forcebase` / `forcephase` / `forcegrad` / `forceenabled` | 旁路稳定门，仅调试 |

单元测试：`control/tests/test_tuner_debug_api.cpp`（13 个用例，含"force 确属旁路" 与"经 API 的 tick 与直接 tick 逐位一致"两条关键保证）。

端到端验收：`tools/tuner_debug_accept.py` 驱动 CLI 对 9 个功能组 49 项断言 （每条命令的预期数值均经手工核算），退出码可直接接入 CI：

```
python3 tools/tuner_debug_accept.py     # 全绿输出 "49 PASS / 0 FAIL"，退出码 0
```

# 调试参数实时监控（tuner_debug_monitor.py）

`tuner_debug_monitor.py` 驱动 `ar_tuner_debug` 进程，按固定周期下发 `status` 命令并解析 `key=value` 输出，用 matplotlib 实时绘制调参器内部状态：增益（当前/基线）、ESC 梯度、 窗口代价 J、实测振荡与载波相位，共 4 个子图滚动显示。

```
# 交互监控（监控期间可直接在终端输入调试命令，原样转发、回显打印）
python3 tools/tuner_debug_monitor.py

# 内置激励演示：建基线 → 喂 ±2° 方波误差 → 每 6 s 一次 settle，无实船即可看到各参数动起来
python3 tools/tuner_debug_monitor.py --demo

# 调整轮询周期/滚动窗口/落盘 CSV
python3 tools/tuner_debug_monitor.py --interval 0.2 --window 120 --log out/dbg.csv

# 无 GUI 环境冒烟/归档：跑固定时长后保存图片退出
PLOT_SIM_AGG=1 python3 tools/tuner_debug_monitor.py --demo --duration 15 --out out/monitor.png
```

| 选项 | 含义 | 默认 |
| --- | --- | --- |
| `--exe PATH` | `ar_tuner_debug` 可执行文件路径 | `../build/tools/ar_tuner_debug` |
| `--interval S` | `status` 轮询周期 (s) | 0.1 |
| `--window S` | 曲线滚动时间窗 (s) | 60 |
| `--demo` | 内置激励序列 | 关 |
| `--log CSV` | status 快照追加写入 CSV | 关 |
| `--duration S` / `--out PNG` | 固定时长后保存退出 | 关 |

说明：监控器与被调试的调参器是同一进程内实例（经管道请求-响应），在船上使用时 `--exe` 指向船上命令通道的封装即可； `--log` 落盘的 CSV 与调试文档 §6 的行协议字段一致。 详细命令语义见 `docs/实船调试接口调用文档.md` 。